Documento de Arquitetura e Requisitos MVP: Sistema de Ponto Assíncrono

1. Visão Arquitetural e Decisões Técnicas

Este documento define a base técnica para o MVP do Sistema de Ponto. A arquitetura foi desenhada com foco em Alta Disponibilidade (AP), tolerância a falhas e interoperabilidade, adotando o modelo de Arquitetura Orientada a Eventos (EDA).

Decisões Core:

- Agnóstico de Stack (Interoperabilidade): A comunicação entre os serviços é feita de forma neutra, permitindo que produtores e consumidores sejam escritos em qualquer linguagem de programação (Java, Python, Go, Node.js, etc.).
- Broker de Mensageria: ActiveMQ.
- Protocolo: AMQP 1.0. A escolha do AMQP previne vendor lock-in e estabelece a fila de mensagens como um contrato agnóstico (API Assíncrona).
- Padrão de Comunicação: Coreografia de Eventos (Fila 1 para persistência -> Fila 2 para notificação).

2. Requisitos Funcionais (RF)

- RF1 - Recepcionar Batida de Ponto (Producer): A API deve receber a matrícula (ID) do funcionário, gerar o timestamp exato e enviar a mensagem para a Fila 1 (Processamento). Retorna 202 Accepted imediatamente.
- RF2 - Persistir Dados (Consumer A): Um worker lê a Fila 1, valida a mensagem e salva o registro no banco de dados (ex: H2/PostgreSQL).
- RF3 - Acionar Notificação (Produtor Interno): Após persistir com sucesso, o Consumer A emite um evento para a Fila 2 (Notificação).
- RF4 - Comprovar Ponto (Consumer B): Um worker lê a Fila 2 e simula o envio de e-mail (log) comprovando a operação.
- RF5 - Consultar Histórico: API GET simples para listar os pontos batidos por um usuário.

3. Requisitos Não Funcionais (RNF)

- RNF1 - Alta Disponibilidade (AP): A API principal (Producer) não depende do banco de dados e deve responder em menos de 100ms.
- RNF2 - Consistência Eventual: Tolerância a atrasos de processamento na fila para atualização da leitura.
- RNF3 - Garantia de Entrega: Uso de Manual ACK (Acknowledge). A mensagem só é removida da fila após processamento bem-sucedido.

- RNF4 - Tolerância a Falhas: Se o banco falhar, a mensagem retorna para a Fila 1 (reprocessamento).

4. Regras de Negócio (RN)

- RN1 - A Verdade é o Timestamp da API: O horário oficial do ponto é o momento da requisição na API, não o da persistência no banco.
- RN2 - Validação Tardia (Lazy Validation): A API não consulta o banco para validar o ID. O Consumer A faz isso de forma assíncrona.
- RN3 - Debounce (Anti-Duplicidade): O sistema ignora batidas de ponto do mesmo ID num intervalo menor que 60 segundos.
- RN4 - Comprovação Estrita: A notificação de sucesso (Fila 2) só ocorre se o dado foi realmente persistido.

5. Design de Contratos (Payloads JSON)

Para garantir a independência de tecnologia, os contratos (JSONs) trafegados no ActiveMQ via AMQP são autodescritivos, utilizando o padrão ISO 8601 para datas e o conceito de Event Enrichment (Enriquecimento de Eventos).

5.1. Payload da API Pública (POST /pontos)

```
{
  "idFuncionario": "98765"
}
```

5.2. Contrato da Fila 1 (RegistrarPontoCommand)

```
{
  "mensagemId": "a1b2c3d4...",
  "idFuncionario": "98765",
  "timestampRegistro": "2026-10-14T20:38:00Z"
}
```

5.3. Contrato da Fila 2 (PontoRegistradoEvent)

```
{
  "mensagemId": "f9e8d7c6...",
  "idFuncionario": "98765",
  "nomeFuncionario": "João da Silva",
  "emailFuncionario": "joao@empresa.com",
  "timestampRegistro": "2026-10-14T20:38:00Z"
}
```

---

6. Diretrizes de Arquitetura de Software (Ports and Adapters)

O domínio da aplicação deve se comunicar com o mundo externo estritamente através de interfaces (Portas). A implementação real da tecnologia ficará na camada de infraestrutura (Adapters).

7. Estrutura do Repositório (Monorepo)

O projeto adota Monorepo para centralizar contratos e facilitar a execução local. A infraestrutura (Docker) fica na pasta /infra, isolada das pastas de cada aplicação.

```
sistema-ponto-mvp/
    docs/              # Documentação (como este DOCX)
    infra/             # docker-compose.yml (ActiveMQ, PostgreSQL)
    api-producer/          # Serviço Java/Spring (Recebe requisição
e enfileira)
    worker-persistence/      # Serviço Java/Spring (Lê Fila 1, Salva
BD, Envia Fila 2)
    worker-notification/     # Serviço Node.js/Python (Lê Fila 2 e
notifica)
```

---

8. Estratégia de Configuração e Ambientes (12-Factor App)

Configurações que variam entre ambientes são injetadas exclusivamente via Variáveis de Ambiente (ENVs), priorizando a Nuvem em PROD/QA e arquivos .env isolados na máquina local em DEV. A aplicação adota a política de "Fail Fast" para ausência de variáveis vitais em produção.

9. Modelagem de Dados (Relacional)

Para manter a simplicidade do ambiente open-source, o MVP utilizará um banco de dados relacional (H2 em memória para DEV, PostgreSQL via Docker para testes/produção).

Tabela: funcionario (Apenas Leitura no MVP)

```
CREATE TABLE funcionario (
  id VARCHAR(50) PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL
);
```

Tabela: registro_ponto (Append-only)

```
CREATE TABLE registro_ponto (
```

```
    id UUID PRIMARY KEY,
    mensagem_id UUID NOT NULL UNIQUE, -- ID da fila (Idempotência)
    id_funcionario VARCHAR(50) NOT NULL,
    timestamp_registro TIMESTAMP WITH TIME ZONE NOT NULL,
    criado_em TIMESTAMP WITH TIME ZONE DEFAULT
CURRENT_TIMESTAMP,
    CONSTRAINT fk_funcionario FOREIGN KEY (id_funcionario)
REFERENCES funcionario(id)
);
```

---

10. Topologia das Filas (Mensageria)

O fluxo principal utiliza duas filas no ActiveMQ. Para esta primeira iteração do MVP, o tratamento de falhas focará na rejeição natural (NACK) com retorno para a fila principal, sem a complexidade de Dead Letter Queues (DLQ).

- Fila 1 (Persistência): fila.ponto.processamento
  Propósito: Receber o comando da API. O Consumer A (Worker Persistence) consome desta fila.
- Fila 2 (Notificação): fila.ponto.notificacao
  Propósito: Receber o evento enriquecido do Consumer A. O Consumer B (Worker Notification) consome desta fila.